

UVM Configuration Intelligence

An intelligent platform for transforming massive Universal Verification Methodology (UVM) simulation logs into ranked root-cause attributions, deterministic RTL bug isolations, multi-objective Pareto trade-offs, and constrained Bayesian recommendations.

1,015,872

LOG LINES PARSED

2.0 s

FULL 50K PARSE TIME

0.88

RISK MODEL ROC-AUC

0.999

RTL BUG DETERMINISM

1. Problem Statement & Motivation

In modern semiconductor and ASIC verification, nightly regressions generate millions of lines of unstructured simulation logs. When runs fail, verification engineers face four severe triage bottlenecks:

⚠️ The "What" vs. "Why" Dilemma

A failed UVM test logs *that* an assertion fired or FIFO overflowed, but fails to explain *which combination of configuration knobs* triggered the failure state.

🔍 Flake vs. RTL Bug Conflation

Random seed-induced noise and hard deterministic RTL bugs emit similar UVM error reports, forcing engineers into days of tedious interactive waveform debugging.

🔧 Black-Box Performance Tuning

Configuration tuning remains folklore. Verification leads lack tools to determine the fastest configuration settings that provably satisfy a failure budget (e.g., $\leq 2\%$).

🗑️ Ad-Hoc, Non-Accumulating Triage

Custom triage scripts are written per debug emergency and discarded immediately after. Verification knowledge never accumulates systematically across builds.

2. End-to-End Pipeline Architecture

The platform executes an asynchronous, modular analytics pipeline. An optimized parser processes log dumps into structured data, feeding five decoupled analytical stages in parallel:



Fault-Isolated Execution: Every stage in `uvmm_intel/pipeline.py` is isolated with exception wrapping. If an optional engine encounters edge-case input, the overall report remains intact and delivers partial analytics with diagnostic traces rather than aborting the pipeline.

3. Key Technical Innovations

1. Leakage-Guarded Risk Modeling

Trained with LightGBM on *only settable configuration knobs*. All outcome variables (`execution_time_ms` , `throughput_mbps` , `uvm_error_count` , tags) and identifiers (`run_id` , `seed`) are strictly pruned, preventing artificial label leakage. SHAP TreeExplainer delivers directional attribution per feature value.

2. Determinism Scoring Engine

Masks dynamic fields (addresses → `<ADDR>` , values → `<N>`) and fits TF-IDF across unique templates. Collapses 50k logs into ~7 clusters. Defines:

`Determinism = 1 - (Distinct Raw Traces / Total Cluster Runs)` .

Isolates true RTL bugs (score 0.999) from random flakes (score 0.000).

3. Top-SHAP Grouped Pareto Frontier

Rather than evaluating continuous sensor readings (voltage, temperature) which treat each run as an isolated singleton, points are aggregated over the **top-4 discrete knobs ranked by SHAP**. Produces 318 meaningful configuration clusters with observed rates tracking predicted risks exactly.

4. Constrained Bayesian Optimization

Optuna TPE searches the multi-dimensional design space against two LightGBM surrogates: risk and throughput. The throughput regressor trains *exclusively on passing runs* to measure true operating capability, enforcing an explicit penalised ≤ 2% risk ceiling.

4. Embedded Ground Truth Validation Benchmark

The synthetic log generator embeds 7 deterministic and statistical interaction rules (defined in `data/uvm_ground_truth.json`). The pipeline's discovery algorithms are benchmarked against this definitive answer key:

Rule	Trigger Condition	Simulated Impact	Runs	Fail Rate	Lift	Verification Objective
R1	<code>test_name = Back_To_Back_Program</code> + <code>cache_policy = Adaptive</code>	P(fail) = 0.85, <code>FIFO_OVERFLOW</code>	2,891	85.1%	4.19×	Multi-parameter corner case
R2	<code>test_mode_enabled = 1</code>	+0.35 Failure Probability	13,999	47.3%	3.16×	Single high-impact dominant knob
R3	<code>queue_depth ≥ 32</code> + <code>ecc_mode = Disabled</code>	+0.35, <code>ECC_UNCORRECTABLE</code>	1,976	57.4%	2.53×	Memory sub-system stress interaction
R4	<code>temperature_c > 85</code> + <code>voltage_mv < 1100</code>	+0.30, <code>RETENTION_FAIL</code>	281	55.9%	2.34×	Rare environmental physical condition
R5	<code>num_planes = 4</code> + <code>burst_length = 64</code> + <code>scrambler_enable = 1</code>	P(fail) = 0.98, Deterministic byte trace	815	98.7%	4.33×	Planted RTL bug (byte-identical)
R6	<code>clock_freq_mhz ≥ 1000</code> + <code>prefetch_depth > 8</code>	+0.30, <code>TIMEOUT</code>	8,014	47.5%	2.43×	Clock & prefetch throughput limits
R7	<code>cache_policy = Disabled</code> + <code>queue_depth ≥ 16</code>	+0.30, <code>DATA_MISMATCH</code>	2,800	49.3%	2.19×	Coherency / buffer overflow bug

5. Interactive Web Application (React 18 + Vite)

The frontend interface (in `frontend/src/`) connects via REST polling to FastAPI, organizing regression insights into 7 views:

1. Executive Summary

KPI cards (Runs, Fail Rate, Error Lines), Pass/Fail donut chart, Risk histogram, global SHAP chart, error taxonomy, and held-out ROC curve.

2. Failure Fingerprints

2D TruncatedSVD scatter projection of failure clusters, determinism table, masked template vs. raw log viewer, and one-click handoff to diff.

3. Trade-off Matrix

Interactive Pareto scatter with highlighted Peak Throughput, Knee, and Best-Safe operating points across 318 discrete configurations.

4. Recommendations

Optuna TPE search results, ranking parameter configurations that achieve peak throughput while staying below the 2% failure risk threshold.

5. Config Diff

Nearest-twin run analyzer comparing failing runs against closest passing counterparts (≥ 0.5 SD change threshold) to spotlight broken settings.

6. Run Explorer & Details

Full virtualized, searchable run table with pagination, field-level lift metrics, pipeline execution stage timings, and one-click JSON export.

6. Repository Structure & Artifacts

Production Platform (`uvm_intel/` & `backend/`)

- `log_format.py` : Canonical UVM log schema.
- `log_parser.py` : High-throughput regex/line parser.
- `risk_model.py` : Leakage-safe LightGBM + SHAP.
- `fingerprints.py` : Text template DBSCAN clusterer.
- `pareto.py` & `recommender.py` : Optuna optimizer.
- `config_diff.py` : Nearest-twin KNN comparator.
- `backend/main.py` : FastAPI service with threaded jobs.

Baseline Discovery Engine (Root Directory)

- `data_model.py` : Schema specifications for test data.
- `generate_dataset.py` : Vectorized NumPy synthetic generator.
- `train_baseline.py` : Random Forest baseline classifier.
- `interaction_analysis.py` : Pairwise/3-way lift & support.
- `grade_against_ground_truth.py` : Precision/Recall grader.
- `ingest.py` : Fallback tolerant parser for messy inputs.
- `presentation/build_deck.js` : Presentation deck builder.

7. Measured System Benchmarks

Corpus Size	Log Volume	Log Parse Time	End-to-End Analytics Time	Memory Footprint
5,000 runs	~5.6 MB (101k lines)	0.6 seconds	9.6 seconds	< 180 MB
10,000 runs	~11.2 MB (203k lines)	1.2 seconds	27.0 seconds	< 260 MB
50,000 runs	~56.0 MB (1,015,872 lines)	2.0 seconds	Wrapped Thread Execution	< 510 MB

8. Roadmap & Production Extensions

- RAG Copilot (Phase 4 scope):** Natural language Q&A over parsed regression archives with ChromaDB + LangChain embeddings.
- Longitudinal Regression Tracking:** Continuous tracking of SHAP importance drift, failure clusters, and risk inflation across nightly builds.
- CI/CD Quality Gate:** Automated pipeline gate that fails validation if a new cluster with determinism score > 0.8 is detected.
- Commercial Simulator Adapters:** Pluggable AST parsers for Synopsys VCS, Siemens Questa, and Cadence Xcelium transcript formats.